

Project brief: machine learning quantum number assignment for water

Marco Barnfield (marco.barnfield.24@ucl.ac.uk), day-to-day supervisor

PIs: Jonathan Tennyson, Sergei Yurchenko

ExoMol group, UCL Physics & Astronomy

October 2026

1 Problem and gap

Variational codes (DVR3D for water, TROVE for polyatomics, Duo for diatomics) compute energy levels very accurately, but each level only comes out with its rigorous labels: total angular momentum J and symmetry. The approximate quantum numbers that spectroscopists actually use, $(v_1 v_2 v_3)$ and (K_a, K_c) for water, have to be assigned afterwards. Without them, a computed level cannot be matched to experiment, replaced with an empirical (MARVEL) energy, or used to build a band-resolved line list.

The most recent attempt for water is Zobov et al. (2026, JQSRT 354, 109833), who labelled the UCLH2O296 room-temperature line list (241,234 states, $J \leq 26$, up to $44,500 \text{ cm}^{-1}$) with a hand-tuned combination of five rule-based methods (wavefunction contribution, node counting, a modified Hose–Taylor method, label merging, and K_a energy-dependence correction). The result:

- 92,035 levels (38%) receive K_a , K_c and v_2 ;
- only 48,440 levels (20%) receive the full label $(v_1 v_2 v_3) J K_a K_c$;
- no learning from experimental (MARVEL W2020) labels, and no per-level confidence;
- not extended to the high-temperature POKAZATEL line list (much higher J and energy).

The companion potential energy surface paper, Rogov et al. (2026, JQSRT 354, 109863), changes where machine learning can help. Their PES reproduces more than 90% of the MARVEL H_2^{16}O levels with $J \leq 15$ to a standard deviation of 0.01 cm^{-1} , and predicts $J = 16\text{--}27$ to about 0.013 cm^{-1} . Any computed level that has a MARVEL counterpart can therefore be labelled by energy matching alone; a classifier adds nothing there. The real problem is the levels with *no* MARVEL counterpart, which make up most of the unlabelled 80%, plus the small fraction the PES misses by more than the tolerance.

As far as we know, no published machine learning method assigns water quantum numbers. This project asks whether a pretrained decision model (Laya), trained on the levels that can be labelled (by energy matching or by Zobov et al.), can extend labels to the levels that have no experimental counterpart, with calibrated confidence.

2 Water notation, minimum needed

- **Vibration** $(v_1 v_2 v_3)$: symmetric stretch, bend, antisymmetric stretch. Because the stretch frequency is roughly twice the bend, levels group into polyads $P = 2v_1 + v_2 + 2v_3$ whose members mix strongly. Above about $10,000 \text{ cm}^{-1}$ the stretches behave as local modes, and normal mode labels become ambiguous. Expect most errors here.
- **Rotation** $J K_a K_c$: water is an asymmetric top, so each J has $2J + 1$ rotational levels labelled by K_a and K_c with $K_a + K_c \in \{J, J + 1\}$. Given J , K_a and the rotational symmetry, K_c is fixed, so the rotational label effectively reduces to K_a .
- **Symmetry**: levels split into ortho and para (nuclear spin), set by the parity of $v_3 + K_a + K_c$, and further into C_{2v} symmetry blocks. A computed level's symmetry block is exact; within one (J, block) every full label can appear at most once.

- **MARVEL**: an empirical network method that inverts measured transitions into energy levels with uncertainties and labels. For H_2^{16}O this is the W2020 database.

3 Background: CO_2 and Laya

The group has assigned quantum numbers to 12 CO_2 isotopologues with a graph neural network (Barnfield, Yurchenko & Tennyson 2026, JQSRT 365, 110111), used in the ExoMol CO_2 line lists (Yurchenko et al. 2025). Three ideas transfer directly to water:

1. **Uniqueness via Hungarian assignment.** A per-level classifier happily gives two levels the same label. Solve a linear assignment problem on cost $1 - p$ within each block of mutually exclusive labels, here (J , symmetry block).
2. **Post-solver confidence.** Use the margin between the assigned label and the runner-up *after* the solver (`assigned_margin`), not the raw argmax margin. It automatically penalises contested levels.
3. **Bootstrapping.** Promote confidently labelled levels into the training set and retrain, generation by generation.

Laya. Laya (ConvAI Innovations, Apache-2.0, <https://huggingface.co/convaiinnovations/laya>) is a ModernBERT-large encoder (421M parameters) with a decision head that answers multiple-choice or yes/no questions about a text “state” with calibrated probabilities, about 33 ms per call. Known limits: accuracy drops sharply above about 20 options, it ships over-confident (refit the temperature), and the English checkpoint has an invalid temperature for choice questions with 11 or more options. **Keep native-head questions to 10 options or fewer.**

On CO_2 , the Laya backbone was LoRA fine-tuned as a plain row classifier (each level serialised as one line of text, no graph). It reached 94.88% exact match on held-out levels, against 94.5–96.9% for the GNN. So the classifier is not the bottleneck for water; the label encoding is, because the CO_2 label scheme has no water equivalent.

4 The project

The project is free form. The steps below are a suggested route, not a schedule.

Suggested warm-up: HD^{16}O computed-to-MARVEL matching. Match computed HD^{16}O levels (DVR3D) to MARVEL HD^{16}O levels by Hungarian assignment on $|E_{\text{calc}} - E_{\text{MARVEL}}|$ per (J , block), then ask Laya’s native head to judge the doubtful pairs (yes/no, or a ≤ 10 -option choice). Isotopologue calculations are less accurate than H_2^{16}O , so energy matching is a real test here. Small, fully checkable, and it builds the data handling used later.

Main task: labelling UCLH2O296.

1. **Energy matching (not ML):** transfer W2020 labels onto UCLH2O296 per (J , block), accepting pairs within a few times 0.013 cm^{-1} . This labels the experimentally known levels and builds the training set. Pairs outside the tolerance form a hard test set.
2. **Classifier:** Laya backbone as a row classifier, followed by Hungarian assignment per (J , block).
3. **Judge:** Laya native head adjudicates low-margin or contested levels, choosing among ≤ 10 candidate labels from the classifier.
4. **Calibration:** temperature-scale probabilities on held-out levels, then pick a confidence threshold that targets a stated error rate.
5. **Optional:** bootstrap generations, as for CO_2 .

Open design choices.

- **Label encoding:** (a) one joint class per $(v_1v_2v_3)K_a$, large and sparse; (b) separate vibrational and K_a heads with K_c derived; (c) no fixed class list, generate candidates and let Laya choose. Option (b) is the suggested start.
- **Features in the text state:** E , J , symmetry block, ortho/para, energy gaps to neighbours in the same block, and the label and energy of the matching level at $J - 1$ (the rotational ladder). Any DVR3D wavefunction diagnostics available are worth trying.
- **Evaluation split:** the labelled levels are biased towards low energy and low J , while the target levels sit mostly above them. Hold out the highest energy bands and highest J of the labelled set, not a random sample, so the test mimics the target.

Stretch: other classifiers. Laya is one choice of pretrained model, not the only one. Worth comparing if time allows:

- **Tabular foundation models** (e.g. TabPFN): take numeric features directly instead of text, which avoids tokenising energies. Best suited to the small candidate sets of the judge step.
- **Small open-weight LLMs** as judges: score each candidate label by log-likelihood instead of generating text.
- **Smaller encoders** (ModernBERT-base, DeBERTa-v3): same recipe as Laya, faster, shows whether model size matters.
- **Gradient-boosted trees** (LightGBM): the non-LLM sanity check every ML result should beat.

5 Success criteria

Ticking off every item gives a *Journal of Quantitative Spectroscopy and Radiative Transfer* paper, working title *Machine learning quantum number assignment for water line lists*. Each item names the paper section it fills.

1. **Dataset** (Data): UCLH2O296 matched to W2020, with the tolerance, counts of matched, unmatched and large-residual levels, and the evaluation split. Released as a file.
2. **Baselines** (Method): energy matching for levels with a MARVEL counterpart; extrapolation of labelled rotational ladders (energy against $J(J+1)$ per vibrational state and K_a) for those without.
3. **Classifier result** (Results): Laya plus Hungarian assignment beats the ladder baseline on the held-out high-energy and high- J levels, reported per energy band.
4. **Calibrated confidence** (Results): a threshold on `assigned_margin` or calibrated probability with a measured error rate on held-out levels.
5. **Coverage** (Results): fraction of UCLH2O296 fully labelled at that threshold, compared with the 20% of Zobov et al., plus agreement with their labels where both exist. The labels are released.
6. **Judge** (Results): whether Laya’s native head improves contested levels, quantified. A negative result counts.
7. **Failure analysis** (Discussion): where and why labels fail (local modes, polyad mixing), and what that means for POKAZATEL and other ExoMol molecules.
8. **Manuscript**: the above written up and submitted.

6 Code, compute and data

Code. Fork https://github.com/mbarnfield63/GNN_CO2_QN_assignment and work on branch `h2o-student`, a clean slate with no CO₂ code. It holds this brief and the two pieces worth reusing:

- `src/assignment.py`: Hungarian assignment per block, with locked (known) labels and `assigned_margin`. Ported from the CO₂ pipeline.
- `src/train_classifier.py`: LoRA fine-tune of the Laya backbone on `{"text", "label"}` rows, plus a `predict` command that writes logits for the assignment step.

Everything molecule-specific (parsing, matching, label encoding, serialisation, evaluation) is yours to write. The CO₂ code is on the repository’s other branches if you want to see how it was done there: the GNN pipeline on `main`, the Laya prototype in `experiments/laya_prototype/` on `CDSD-Analysis`.

Compute. Install `uv`, then run `uv sync` in the repository. Training needs an NVIDIA GPU with at least 8 GB (RTX 20-series or newer); on CO₂ it peaked at 5.4 GB and took about 75 minutes on an RTX 2070. Inference needs about 2 GB. Without a suitable GPU, use a free T4 on Google Colab (<https://colab.research.google.com>, Runtime → Change runtime type → T4 GPU).

Data.

- UCLH2O296 labelled levels: Zobov et al. (2026), open access; [check supplementary data availability, otherwise ask supervisor](#).
- MARVEL W2020 H₂¹⁶O levels: supplementary material of Furtenbacher et al. (2020).
- POKAZATEL (stretch): <https://www.exomol.com/data/molecules/H2O/1H2-16O/POKAZATEL/>.
- From the supervisor: HD¹⁶O computed and MARVEL levels for the warm-up.